UGANDA CHRISTIAN UNIVERSITY

A Centre of Excellence in the Heart of Africa

CSC2107

DESIGN & ANALYSIS OF ALGORITHMS

Assignment 5.1

Naive vs. Memoized Fibonacci: Implementation, Experiments and Complexity Analysis

GROUP 17 MEMBERS

No.	Full Name	Registration Number
1	Ainebyoona Ron	M23B23/041
2	Osengor Jonathan	S23B23/052
3	Kocho Max	S25B23/112
4	Ainembabazi Ninah	M25B23/055
5	Martin Kananura	M22B23/010
6	Sanyu Shakira	S25B23/092

Submission date: 22 October 2026

Overview

In this assignment we compared two ways of finding the n-th Fibonacci number. Algorithm A is plain recursion. Algorithm B is also recursion, but it remembers every answer it has already worked out (this is called memoization). We wrote both in Python and added a counter to each one that counts how many times the function is called. We timed every run with time.perf_counter(). We also checked that both algorithms give the same answer for every n from 0 to 20, and they do.

In short: Algorithm B is far faster. Its work grows in a straight line with n, while Algorithm A's work explodes as n gets bigger. The numbers below show how big the gap is.

Note: the call counts will be the same on any computer, but the times in seconds depend on the machine used.

Key findings at a glance

Measure	Algorithm A (plain)	Algorithm B (memoized)
Calls for n = 35	29,860,703	69
Calls for n = 1000	$\approx 1.4 \times 10^{209}$ (estimated)	1,999
Time for n = 1000	$\approx 10^{194}$ years (estimated)	0.0004 s (measured)
Time complexity	Exponential, $\Theta(\phi^n)$	Linear, $\Theta(n)$
Space complexity	O(n)	O(n)

Table 0: Summary of the comparison (details and proofs are in Parts 2 to 5).

Part 2: Experimental Results

2.1 Algorithm A (plain recursion)

n	calls	time (s)
5	15	0.000002
10	177	0.000012
15	1,973	0.000121
20	21,891	0.001397
25	242,785	0.015773
30	2,692,537	0.182553
35	29,860,703	2.225130

Table 1: Algorithm A. No run took longer than 60 seconds, so the largest n we reached was 35, which is the last value in the assignment.

2.2 Algorithm B (memoization, fibtable cleared before each run)

n	calls	time (s)
10	19	0.000007
50	99	0.000015

n	calls	time (s)
100	199	0.000034
200	399	0.000085
500	999	0.000157
1000	1,999*	0.000402

*Table 2: Algorithm B. *For n = 1000 we had to raise Python's recursion limit first (explained in Part 4).*

2.3 Graph of n against the number of calls

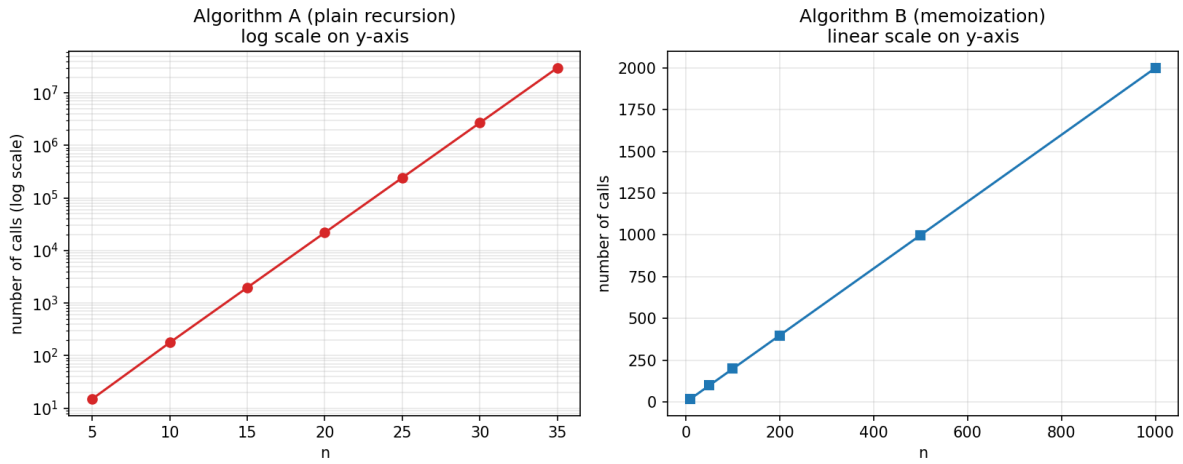


Figure 1: Left: Algorithm A with a log scale on the y-axis. Right: Algorithm B with a normal scale.

Why we used a log scale for Algorithm A. The number of calls for A goes from 15 all the way to almost 30 million. On a normal axis, every point except the last one would be squashed against the bottom and the graph would be useless. A log scale gives each power of 10 the same amount of space, so we can see all the points clearly.

A log scale also helps us spot exponential growth, because exponential growth shows up as a straight line on it. Our graph for A is a straight line, so A grows exponentially. We can also see how fast: when n goes up by 5, the calls get multiplied by about 11 (29,860,703 ÷ 2,692,537 ≈ 11.09). Since $1.618^5 \approx 11.09$, this means the calls grow by about 1.618 times every time n goes up by 1.

For B, the graph is a straight line on a normal scale, so the calls grow in a straight line with n. The line goes up by 2 calls for every 1 step in n.

Part 3: Analysis

3.1 Recurrence relations for the number of calls

Algorithm A. When n is 2 or more, a call counts itself (1) and then makes one call for n − 1 and one call for n − 2. When n is 0 or 1, it makes no more calls, so it counts as just 1.

$$T_A(n) = T_A(n-1) + T_A(n-2) + 1 \quad \text{for } n \geq 2, \quad T_A(0) = T_A(1) = 1$$

Algorithm B (starting with an empty table). A call to fib(n) counts itself (1). It then calls fib(n−1), which does all the real work and makes T_B(n−1) calls. After that, it calls fib(n−2). But the answer for n−2 was already saved in the table while fib(n−1) was running, so this call costs just 1 call and returns straight away.

$$T_B(n) = T_B(n-1) + 2 \text{ for } n \geq 2, \quad T_B(1) = 1$$

3.2 Algorithm A: calls(n) = 2·F(n+1) − 1, and why it grows exponentially

What we want to show: T_A(n) = 2·F(n+1) − 1 for all n ≥ 0. We use induction.

- **Base cases:** T_A(0) = 1 and 2·F(1) − 1 = 2 − 1 = 1, so it works. T_A(1) = 1 and 2·F(2) − 1 = 2 − 1 = 1, so it works too.
- **Step:** assume the formula is true for n − 1 and n − 2. Then

$$T_A(n) = T_A(n-1) + T_A(n-2) + 1 = (2F(n) - 1) + (2F(n-1) - 1) + 1$$
$$= 2(F(n) + F(n-1)) - 1 = 2 \cdot F(n+1) - 1. \blacksquare$$

The last step works because F(n) + F(n−1) = F(n+1) by the definition of Fibonacci numbers.

We also checked this with our data. For n = 10, 2·F(11) − 1 = 2·89 − 1 = 177, which matches Table 1. For n = 35, 2·F(36) − 1 = 2·14,930,352 − 1 = 29,860,703, which also matches. Our program checked the formula for every n from 0 to 25 and it was right every time.

Why this means exponential growth. The Fibonacci numbers grow like $F(n) \approx \phi^n / \sqrt{5}$, where $\phi = (1 + \sqrt{5}) / 2 \approx 1.618$. So

$$T_A(n) = 2 \cdot F(n+1) - 1 \approx (2/\sqrt{5}) \cdot \varphi^{n+1} = \Theta(\varphi^n)$$

Each call does only a small, fixed amount of work, so the running time grows in the same way: $\Theta(\phi^n)$, which is about 1.618^n. It is often written as the simpler upper bound $O(2^n)$, because every call splits into two more calls. The real growth rate is a little smaller than 2 because the n − 2 branch of the call tree is shorter than the n − 1 branch.

3.3 Algorithm B makes 2n − 1 calls

What we want to show: T_B(n) = 2n − 1 for n ≥ 1 when the table starts empty.

- **Base case:** T_B(1) = 1 = 2·1 − 1. (As a further check, fib(2) makes 3 calls: fib(2), fib(1) and fib(0).)
- **Step:** if T_B(n−1) = 2(n−1) − 1 = 2n − 3, then T_B(n) = T_B(n−1) + 2 = 2n − 1. ■

A simple way to see it: every value from F(0) up to F(n) gets worked out exactly once, which gives n + 1 calls. The only other calls are table look-ups: for each k from 3 to n, the call fib(k−2) finds its answer already saved, which gives n − 2 more calls. In total (n + 1) + (n − 2) = 2n − 1. Our experiment matched this exactly for every n we tried (19, 99, 199, 399, 999 and 1999 calls), and our program also checked every n from 1 to 200.

Time complexity: every call does a small, fixed amount of work, so the total time is Θ(n). **Space complexity:** the table holds n + 1 values, which is O(n), and the recursion goes n levels deep, which is also O(n). So the space is O(n) overall. (To be exact, F(n) has about 0.2n digits, so adding big numbers is not completely free. This does not change the main comparison.)

3.4 Complexity class of each algorithm

	Algorithm A	Algorithm B
Recurrence	T(n) = T(n−1) + T(n−2) + 1	T(n) = T(n−1) + 2
Closed form	2·F(n+1) − 1	2n − 1
Time class	Exponential, $\Theta(\phi^n) \subseteq O(2^n)$	Linear, Θ(n)

	Algorithm A	Algorithm B
Space	O(n) (stack depth only)	O(n) (table + stack)
Data evidence	Straight line on log graph; calls × 11.09 for every +5 in n (= ϕ^5)	Straight line on normal graph; 2 extra calls for every +1 in n

From the recurrences: A's recurrence adds together two smaller problems that are almost the same size as the original. This always leads to exponential growth. B's recurrence only adds a fixed number (2) for each step down, which leads to linear growth.

From the data: the experiments agree. In A, the time gets multiplied by about 11 each time n goes up by 5 (0.0158 s, then 0.183 s, then 2.23 s). In B, we reached n = 1000 in less than a millisecond.

3.5 What would Algorithm A need for n = 1000?

- Calls needed: $2 \cdot F(1001) - 1 \approx 1.41 \times 10^{209}$.
- Speed we measured at n = 35: 29,860,703 calls in 2.225 s, which is about 1.34×10^7 calls per second.
- Time needed: $1.41 \times 10^{209} \div 1.34 \times 10^7 \approx 1.0 \times 10^{202}$ seconds, which is about 3.3×10^{194} years.

Is it feasible? No, not at all. The universe is only about 1.4×10^{10} years old, so this would take around 10^{184} times longer than the age of the universe. The number of calls (10^{209}) is even bigger than the number of atoms in the observable universe (about 10^{80}). A faster computer would not help. Even one a trillion times faster would make no real difference.

Part 4: Pushing to n = 1000

4.1 What happens with Python's default settings

What happens: Algorithm B crashes at n = 1000 with the error `RecursionError: maximum recursion depth exceeded`. Our program printed sys.getrecursionlimit() and got 1000.

Why: Memoization cuts down the number of calls, but it does not make the recursion shallower. fib(1000) calls fib(999), which calls fib(998), and so on all the way down to fib(1) before any of them can return. That means about 1000 function calls are waiting on the stack at the same time. Together with the frames of our own script, this is more than Python's default limit of 1000. Python has this limit to stop the program from using up its stack memory. At n = 500 there is no problem, because 500 levels fit inside the limit.

4.2 Two fixes and their trade-offs

Fix 1: raise the recursion limit.

```
import sys
sys.setrecursionlimit(5000)   # allow deeper recursion
fib_B(1000)                   # now works: 1999 calls, 0.0004 s
```

- **Good:** it is just one line, and the algorithm stays exactly the same as the pseudocode.
- **Bad:** the stack still gets n levels deep, and it is still limited by the computer's memory. If we set the limit too high, Python can crash completely instead of showing a clean error. So we have to pick a limit that suits the biggest n we plan to use.

Fix 2: use a loop instead of recursion (bottom-up).

```
def fib_iterative(n):
    if n < 2:
        return n
    previous, current = 0, 1          # F(0), F(1)
    for _ in range(2, n + 1):
        previous, current = current, previous + current
    return current
```

- **Good:** there is no recursion limit problem for any n. It takes Θ(n) time and only O(1) extra space, because it keeps just two variables instead of a whole table. It was also faster in our test (0.000035 s against 0.000402 s for n = 1000), because there are no function calls to pay for.
- **Bad:** it is no longer really memoization. It works out every smaller value in a fixed order, even ones we might not need, and it does not save anything for later use unless we add a table. For harder problems, it can be difficult to find the right order to fill in the answers, while top-down memoization finds the order by itself.

4.3 The check "if fibtable[n] exists" and the value 0

Yes, there is a value where the check can fail: n = 0. F(0) is 0, and 0 is stored in the table. But if "exists" is written as a true/false test (for example if fibtable.get(n): in Python), then 0 counts as False. The saved answer then looks like it is missing.

- **Correctness is not affected.** When the entry looks missing, the function just works out fib(0) again and gets 0 again. We tested a deliberately "buggy" version for n = 10, 100 and 1000, and every answer matched the correct version.
- **Complexity is not affected.** Working out fib(0) again takes a tiny, fixed amount of time and makes no more calls. Also, fib(0) is only asked for by fib(2), and fib(2) runs just once. So the number of calls did not change at all: 19, 199 and 1999 for both versions.

The correct way to write it: check whether the key exists, not whether the value is true. In Python that is `if n in fibtable:` (or `is not None` when using a list). This is a good lesson: a stored value of 0 must never be confused with "nothing stored". In a different problem where a 0 answer is asked for many times, this bug could throw away the savings from memoization.

Part 5: Conclusion

Algorithm B (memoization) is much more efficient. Algorithm A needs $2 \cdot F(n+1) - 1$ calls, which is exponential, $\Theta(\phi^n)$. Algorithm B needs exactly $2n - 1$ calls, which is linear, $\Theta(n)$. The gap is already big at n = 35, where A makes 29,860,703 calls and B makes only 69. That is about 430,000 times more. At n = 1000 the gap is enormous: A would need about 1.4×10^{209} calls (around 10^{194} years on our machine), while B needs 1,999 calls and finishes in under a millisecond. That is about 7×10^{205} times fewer calls. B has one weakness, which is its recursion depth. It fails at n = 1000 with Python's default limit, but this is easy to fix by raising the limit or by using a loop.

The main idea: overlapping subproblems. Fibonacci keeps asking the same smaller questions again and again. For example, fib(n−2) is needed by fib(n) and also by fib(n−1). Plain recursion forgets the answer and works it out again every time, and these repeats multiply at every level. Memoization saves each answer the first time and reuses it, so each different subproblem is solved only once. There are only n + 1 different subproblems (F(0) up to F(n)), so the work drops from exponential to

linear. Memoization only helps when a problem has overlapping subproblems. If all the subproblems are different, as in merge sort, there is nothing to reuse.

(Word count of Part 5: about 238.)